From WebAssembly Bytes to Behavioral Theorems: Exact-Artifact Verification and Proof-Guided Generation in Lean

Anonymous manuscript

Research prototype, 7 August 2026

Abstract

Compiler verification relates a source-language semantics to generated target code, while translation validation checks a particular source-to-target translation. This paper examines a different boundary: proving a functional property directly about one identified WebAssembly binary, without using its source program or compiler in the final theorem. LeanExe implements this boundary in Lean 4 by embedding the complete binary, decoding it with a proved-sound restricted decoder, validating it with a proved-sound validator, translating the validated module to the Talos execution model, and proving equality with the model used by a behavioral weakest-precondition proof.

The companion `leanexegen` tool accepts a prose request and uses separate headless coding-agent sessions to produce a formal specification, a Lean program, and an artifact proof. Compiler annotations describe generated instruction regions and select reusable proof-kit theorems, but Lean checks exact region equalities against the decoded artifact before those theorems can establish behavior. The source program, compiler, annotations, language-model output, and proof-search journal are absent from the logical premises of the final artifact theorem.

The implemented artifact gate covers twenty frozen binaries and a restricted no-import WebAssembly profile. Four generation case studies expose proof-generation costs and the effect of reusable semantic regions: a held-out bounded array-map proof fell from 2364.735 seconds to 103.123 seconds after the library gained a general allocator theorem, a parameterized map-wrapper theorem, an exact compiler annotation, and a deterministic starter. These measurements are development evidence rather than a general benchmark; we designed the final map theorem after inspecting the held-out failure, and the proving agent remains a source of substantial runtime variance. We state the present trust boundary, report an imported-memory defect in the pinned Talos semantics and a known defect in the pinned Lean kernel, and give a roadmap toward broader artifact profiles, independent semantic validation, proof-generating verification conditions, and optional source-theorem transport.

1 Introduction

WebAssembly is a typed, structured bytecode with a specified binary format, validation relation, and operational semantics [7, 29]. Those properties make a WebAssembly binary a plausible subject for independent formal verification, but a theorem about a convenient syntax tree or a compiler's internal module does not identify the bytes that a user executes. The missing link can occur in parsing, validation, serialization, model generation, packaging, or the command that associates a proof with a file.

LeanExe addresses that link by starting its artifact theorem from a Lean value containing every byte of one `.wasm` file. A successful theorem supplies decoded and validated witnesses, a proof of an independent validity judgment, equality between the validated module's Talos translation and

an execution cache, and a behavioral theorem for that module. The compiler and source language do not appear in this implication, so the same verifier architecture applies to any binary inside the accepted profile if a behavioral proof is available.

Direct target-level proof avoids trusting a compiler, but it usually transfers a large amount of work to the artifact proof. Generated WebAssembly includes stack manipulation, local-variable traffic, structured branches, memory-address normalization, allocation code, and ownership operations that obscure application semantics. The central engineering question in this work is whether machine-checked, compiler-informed proof structure can remove that repeated work without admitting compiler claims as axioms.

The current answer combines three components. First, a proof kit states semantic theorems for recurring artifact regions such as array reads, allocator windows, result construction, fixed search chains and trees, and a bounded array-map loop. Second, compiler-produced annotations identify candidate regions, while an independent consumer matches them against the frozen decoded program and emits Lean equalities that reduce to exact instruction templates. Third, `leanexegen` gives those checked equalities and selected theorems to a coding agent that runs Lean during proof construction, after which a separate verifier rebuilds the accepted theorem without the agent or compiler.

This paper makes four contributions as an implementation report. It specifies an exact-byte theorem chain that keeps compiler artifacts, generated caches, and proof hints outside the trusted conclusion. It describes a source-free artifact-proof task embedded in a broader prose-to-specification-to-program workflow. It presents a checked annotation and proof-recipe design that turns compiler provenance into proof selection without trusting provenance. It reports the current twenty-artifact gate, semantic conformance evidence, and controlled proof-generation experiments, including results that failed to improve time.

The implemented system remains a research prototype. Its binary decoder and validator accept a deliberate subset of WebAssembly Core 3.0, its behavioral semantics come from a pinned Talos revision, and its current generated interface centers on flat arrays of unsigned 64-bit words. The repository also pins Lean 4.31.0 despite a known kernel-unsoundness reproduction, supported only by a narrow lexical audit that does not repair the kernel. The paper therefore states results relative to the identified implementation and assumptions rather than claiming WebAssembly-wide or foundational soundness.

2 Goals and theorem boundary

The primary goal is a portable statement about an exact executable artifact. For a user-supplied behavioral predicate S, the final theorem should establish S for the validated execution model recovered from the embedded bytes. Verification should continue to work after removing the Lean source program, LeanExe compiler, WAT renderer, coding agent, annotations, and generation workspaces.

This goal differs from source correspondence. The current artifact theorem establishes the selected specification directly, but it does not prove that the generated source implements the prose request or that the compiler preserves the source function. The specification is a human-review boundary, and the system prints warnings about this distinction when generation succeeds.

For the current array interface, a specification defines

$$\mathit{expected} : \mathsf{Array\ UInt64} \to \mathsf{Array\ UInt64}.$$

The public artifact predicate quantifies over a host environment, initial store, represented input array, and input pointer. Under an allocator-ready initial-state predicate, it requires terminating

invocation of the exported `compute` function and a final-memory array representation equal to *expected*(*input*).

The theorem includes machine-state details that the source theorem omits. The array representation fixes an eight-byte length word followed by one eight-byte word per element, while the readiness predicate fixes allocator globals, places the bump pointer above the input, and supplies page and address bounds for the result. These premises make allocation and memory accesses explicit instead of hiding them in a host shim.

The exact-byte theorem has the following schematic form. The repository declarations use its binary syntax, validator, and Talos module types. The existential witnesses prevent an unchecked cache from serving as the theorem subject.

```
theorem artifact_correct :
  exists raw validated,
    decode artifactBytes = ok raw /\
    validate raw = ok validated /\
    CoreValid raw /\
    ArtifactSpec validated.toTalos
```

The external file-to-value association remains an operational step. The package embeds all bytes in Lean, records their length and SHA-256 digest, and requires the verification command to compare the supplied file with both the frozen package and embedded value. The hash identifies the artifact and protects package handling, while the formal theorem concerns the full byte sequence rather than a cryptographic assumption.

3 System architecture

3.1 Compilation

LeanExe compiles a restricted, pure, monomorphic, first-order subset of Lean 4 to standalone WebAssembly [6, 9]. It imports a kernel-checked declaration from a built Lake module, classifies the reachable call graph, lowers accepted expressions to a typed first-order intermediate representation, and emits a WebAssembly module without the Lean runtime. The intermediate representation has an executable reference interpreter, while the backend produces structured WebAssembly instructions and their binary encoding.

The source subset includes fixed-width unsigned integers, bounded natural numbers, byte arrays, flat arrays, products, selected structures and inductives, and recognized recursion or loop forms. Generated modules use one linear memory, an arena allocator, reference-counted heap headers, and compiler-managed releases for supported ownership patterns. Effects, unsafe or partial declarations, executable axioms, surviving higher-order values, arbitrary runtime type-class dictionaries, and public recursive data lie outside the accepted language.

Compilation supplies artifacts for proof generation but does not justify them. Differential tests compare compiled executions with standard Lean, and a source-driven Talos gate regenerates current compiler outputs before checking specifications. The exact-artifact gate described here does neither: it starts with frozen binary packages and does not invoke the source compiler.

3.2 Generation orchestration

`leanexegen` turns a prose request into a formal specification, a Lean source program, a frozen WebAssembly artifact, and an artifact proof. It runs three separate ephemeral coding-agent sessions

for specification, program, and proof generation. Each session starts in a task-specific temporary workspace with sandboxed write access and invokes real Lean or compiler checks, while the outer orchestrator repeats each final check in a separate workspace.

The formal-specification task context contains the request and defines *expected*. The source-program task context adds the frozen formal specification, writes `compute`, and iterates through Lean type checking, LeanExe subset reporting, and scratch compilation. The artifact-proof task context contains the request, formal specification, frozen Talos program, deterministic artifact modules, selected proof guidance, checked annotation recipes, and the allowed proof-kit catalog; it omits the generated source module, and the proof task does not invoke the compiler.

Figure 1 shows the generation and verification paths. The upper path contains untrusted or review-dependent production steps, including the coding agent and compiler. The lower path identifies the logical subject and rebuilds its theorem from exact bytes.

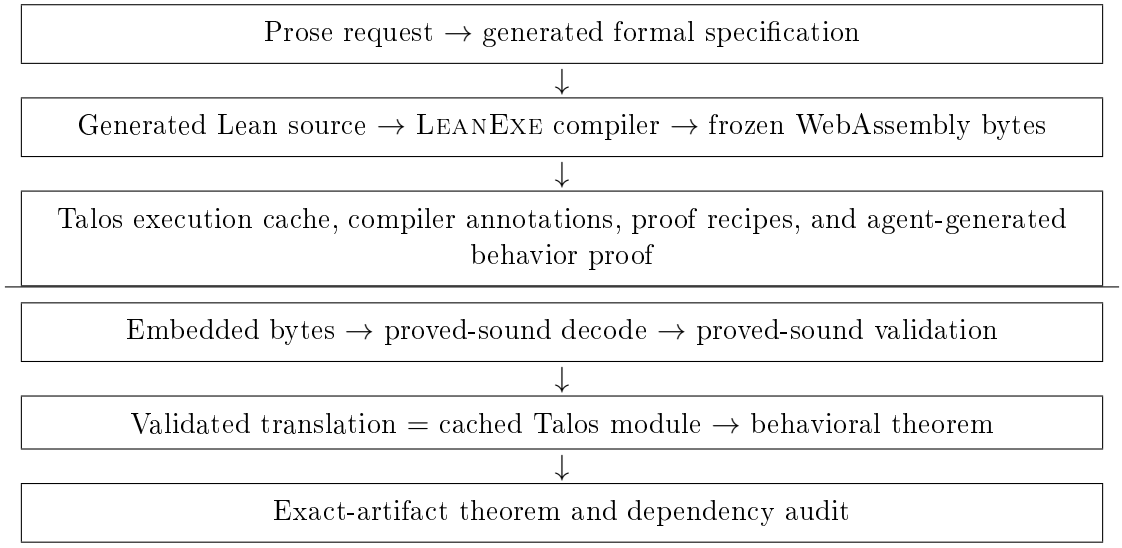

Figure 1: Generation above the line and independent exact-artifact verification below it. Objects above the line can propose proof structure, but the checked conclusion follows from embedded bytes, formal definitions, and Lean proofs.

3.3 Published proof package

A successful run publishes the requested `.wasm` file and a content-indexed proof directory. The directory contains its own byte-identical artifact, formal specification, generated source for inspection, Talos program cache, behavioral proof, deterministic decoder and translation modules, exact-byte checker, task reports, samples, host assumptions, annotations, proof recipes, proof-kit catalog, tool pins, and a proof journal. Independent verification checks every recorded digest and identity before building the theorem.

The source and journal belong to provenance rather than the theorem dependency closure. Package verification creates fresh declaration and byte checkers, audits imports, rejects `sorryAx`, and permits only the repository's recorded logical axioms and checked decision-certificate families. The command requires the pinned Lean and Git dependencies but requires neither coding-agent authentication nor the LeanExe compiler.

The package architecture resembles proof-carrying code in placing code and machine-checkable evidence together [2, 16]. Its policy differs from classic PCC because the theorem can express

application-specific functional behavior and its current check may be expensive. The package also retains extensive generation provenance, although only the exact bytes, formal declarations, permitted proof library, and proof terms enter the logical result.

4 Exact-artifact verification

4.1 Binary decoding and validation

The artifact verifier defines a raw syntax for the accepted WebAssembly profile. The profile covers the type, function, memory, global, export, and code sections; integer and structured-control instructions emitted by LeanExe; one memory; mutable scalar globals; and no imports, tables, start function, element section, or data section. The decoder rejects unrecognized sections, opcodes, features, and module shapes.

The executable decoder uses an explicit byte cursor, checks the magic and version, enforces section order and uniqueness, honors declared section and function-body sizes, parses structured control recursively, and requires complete input consumption. Its specification is a separate declarative relation $\mathsf{Encodes}(bytes, raw)$. A compositional theorem establishes that successful complete-file decoding implies this grammar relation.

Validation remains separate from decoding. An executable validator checks type indices, function and code agreement, local and global indices, mutability, memory presence and limits, export uniqueness, calls, branches, block result types, memory operations, operand-stack effects, and function results. The theorem validate_sound constructs an independent CoreValid judgment from every successful validator result.

This restricted implementation follows WebAssembly Core 3.0 and binary format version 1 [29]. Acceptance is intentionally incomplete: a valid standard module outside the profile is rejected. Sound rejection matters because silently translating an unsupported construct would make every later behavioral theorem depend on an incomplete parser or checker.

4.2 Translation and cache equality

Behavioral proofs use the WebAssembly interpreter and weakest-precondition library from Talos at revision `bb3277e2...` [5]. Talos can generate a Lean module value from WAT, which is convenient for human inspection and proof development but is not an acceptable exact-byte boundary. A stale or defective WAT renderer or model generator could otherwise cause a proof to concern a different module.

The verifier translates only a validated raw module to the Talos representation. Per-artifact Lean modules cache decoded values and the WAT-derived execution module to control elaboration cost, but checked equalities connect the embedded bytes to the raw cache and the validated translation to the execution cache. The cache therefore affects proof-checking performance without supplying an assumption.

The closed equality is the join point between binary verification and behavioral proof. A behavioral theorem can be developed against readable generated declarations, while the final theorem rewrites that module to the translation recovered from bytes. Removing the compiler, WAT file, and WAT-to-Lean generator after package creation leaves this argument intact.

4.3 Behavioral proof

Talos provides an executable semantics and weakest-precondition rules for WebAssembly instructions. A typical theorem resolves an exported function, quantifies over the initial store and inputs, and proves a `TerminatesWith` postcondition. Artifact-local proofs use symbolic execution, arithmetic, representation predicates, loop invariants, allocator facts, ownership theorems, and memory-frame lemmas.

The shared repository library contains generic proofs for the four runtime functions emitted with each module: allocation, reset, retain, and release. It also contains a recursive ownership-tree theorem for release, read-over-write tactics, address-normalization lemmas, fixed-array representation rules, and application-independent control-flow scaffolds. The twenty frozen artifact proofs reuse these results for scalar functions, byte-array programs, allocation and free-list behavior, recursive release, fixed-array algorithms, and a central-limit-order-book workload.

The formal result is relative to Talos semantics. The repository has no proof that the pinned Talos execution relation refines or equals the normative WebAssembly semantics, so official-corpus and Wasmtime comparisons supply empirical evidence rather than theorem premises. Section 7 discusses a concrete discrepancy and the resulting no-import boundary.

5 Checked annotations and reusable proof regions

5.1 Why annotations are untrusted

Generated WebAssembly contains stable semantic regions that the source compiler knows when it emits them. Examples include a fixed-array length dispatch, indexed element load, equality or less-than search node, result-array allocation, runtime call, counted loop, and complete bounded map wrapper. Rediscovering those regions from raw instruction lists consumed much of the coding agent's time in the first direct proofs.

The compiler can report each region's structured instruction path, half-open interval, local roles, constants, branch roles, and generator identity. Those fields help a proof consumer select a theorem and explain which compiler definition emitted the code. A compiler defect can make any field false, so the annotation cannot enter the theorem as a hypothesis.

The consumer validates artifact identity and resolves each structured path against the frozen decoded Talos program. A kind-specific matcher checks instruction constructors, local indices, constants, operand order, branch bodies, and continuation shape. It then generates an artifact-specific Lean equality between the resolved region and a proof-kit template; Lean must close that equality, usually by reduction, before a semantic recipe can apply.

This design treats annotations as proof-search hints with checked consequences. Incorrect metadata causes a failed match or failed equality rather than an unsound theorem. The final behavior proof depends on the equality and generic semantic theorem, while annotation JSON and compiler provenance can be deleted after proof generation.

5.2 Proof kit and recipe selection

The proof kit records semantic boundaries that recur across artifacts. Its current fixed-array modules cover array representation, indexed loads, length dispatch, allocator windows with shifted and trailing locals, one- and two-word results, saved search frames, equality and less-than nodes, search chains, search trees, and bounded map-add wrappers. Runtime and common modules cover allocation, retain, release, memory frames, calls, blocks, loops, and arithmetic normalization.

A versioned recipe maps each validated region kind to an exact theorem or tactic, supporting declarations, expected postcondition shape, and focused guidance. Recipe selection remains deterministic once the matcher has accepted the region. The generated proof starter imports only the modules required by selected recipes, which reduces both theorem-selection work and accidental dependency growth.

The largest reductions came from semantic composition rather than shorter tactic spelling. A search-chain descriptor represents ten first-match equality nodes in Demo 2, and one inductive theorem proves the complete chain. A search-tree descriptor represents Demo 3's equality and unsigned-order branches, while a wrapper theorem composes length checking, invalid result, query load, tree traversal, and public return.

Demo 4 extends this approach to a loop. The parameterized map-wrapper theorem proves a canonical bounded map for arbitrary maximum length and unsigned addend, including capacity normalization, allocation, result-length storage, a transformed-prefix invariant, payload writes, and the oversized-input empty result. A whole-function `leanexe.array.map-add.v1` annotation is useful only when a checked equality identifies the decoded function with that canonical program.

5.3 Agent feedback and journals

The artifact-proof agent receives the exact build command and can iterate from Lean diagnostics. This follows the broader observation that proof-assistant interaction and premise selection are central to machine-assisted proving [30, 31]. Lean's kernel checks the result regardless of how the agent proposed it, while deterministic tactics such as Aesop illustrate the separate role of auditable proof search inside Lean [11].

`leanexegen` also asks the proof agent to maintain a free-form Markdown journal after Lean checks and changes of direction. The journal is diagnostic evidence rather than proof evidence. Developers read repeated failures from these journals and determine whether a missing semantic theorem, representation lemma, tactic interface, annotation, or generated composition caused the cost.

This process produced several current abstractions. Demo 3's journal showed seven minutes spent reconstructing a graph already implicit in checked region paths, leading to generated tree composition. Demo 4's baseline journal isolated an allocator-frame mismatch, bounded length dispatch, capacity arithmetic, dynamic length store, and transformed-prefix loop, which led to the allocator-window and map-wrapper theorems.

6 Evaluation

6.1 Questions and method

The evaluation asks three questions. The first is whether the artifact-only gate identifies exact bytes and rebuilds meaningful behavior theorems without source or compiler access. The second is how well the accepted profile agrees with official WebAssembly tests and Wasmtime. The third is whether checked annotations and reusable semantic regions reduce the elapsed time required to generate a new behavior proof.

The artifact gate uses twenty frozen binary packages whose theorems already existed against Talos execution models. The exact-artifact migration added embedded bytes, decoding, validation, translation equality, manifest checks, and dependency audits without weakening those behavior statements. The aggregate command starts from the artifact registry and proof workspace and invokes neither LeanExe nor `wasm-tools`.

Proof-generation time is measured from the start of `leanexegen` Stage 5 to the first accepted proof. This interval includes the coding-agent session, its Lean checks, orchestration overhead, and an independent outer acceptance check. Controlled reproofs keep the formal specification, source, frozen bytes, Talos program, deterministic artifact modules, toolchain, and host assumptions fixed while changing the proof kit, annotations, recipes, guidance, or starter.

The timing experiments ran on one resource-limited local lane with one Lean process at a time. Several variants have three-run medians, while the held-out Demo 4 iterations each have one accepted measurement. The retained records identify the Codex CLI version but not the served model, reasoning setting, or a complete hardware specification. Coding-agent search caused large variance in earlier series, so these values document development behavior rather than a reproducible performance comparison.

6.2 Exact artifacts and semantic conformance

All twenty registered packages passed the aggregate exact-artifact gate on 3 August 2026. The cases include scalar arithmetic, input-generic byte scanning, fresh allocation, free-list reuse, reference counting, recursive release, array folds, unsigned LEB128, association-list and order-book queries, and larger order-book transformations. Each manifest names exact identity, decode, validation, translation, behavior, and final artifact declarations.

The repository's broader execution suite reported 791 accepted cases, 45 expected source rejections, 14 expected traps, 340 standard-Lean comparisons, 62 intermediate-representation comparisons, and 56 fuzz cases. These tests evaluate the compiler and host paths but do not participate in the exact-artifact theorem. They remain important because a useful system needs both formal artifact claims and evidence about its unverified production workflow.

The conformance gate pins the official testsuite, Talos, `wasm-tools`, and Wasmtime 44.0.0 [4, 28]. Across twenty-five selected official files, Talos produced 3853 passes, six configured assertion failures, and 627 skips, with no decoder errors, interpreter errors, cascades, or fuel exhaustion. Wasmtime passed all twenty-five files under the selected feature setting.

The same gate extracts fifteen official malformed or invalid modules and requires exact decoder or validator error categories. The cases cover headers, section structure, integer overflow, memory alignment and limits, stack underflow, and stack-height disagreement. These results test the executable checker independently but do not replace the decoder- and validator-soundness theorems.

6.3 Proof-generation case studies

Table 1 summarizes the four principal generation cases. Demo 1 began as a scalar prime-factor counter and later motivated array-wrapper, allocator, call, and loop support in separate fixed-artifact benchmarks. Demos 2 and 3 use the current flat-array interface, while Demo 4 was chosen as a held-out loop case after developing the search-oriented library on the first three demos.

Demo 2's original 1639-line proof took 3260.962 seconds. Shared allocation, indexed-load, and pair-result theorems reduced intermediate runs, but some shorter proofs took longer: a 491-line journal proof required 1407.477 seconds in one run and 365.974 seconds in another. After generated chain composition and a deterministic complete starter, two three-run series had medians of 110.332 and 114.390 seconds, with unchanged 76-line accepted proof source and no agent edit.

Demo 3's original 1398-line proof took 2427.376 seconds. An indexed loader reduced one run to 718.422 seconds and pair-result abstraction reduced another to 278.656 seconds, while a shorter 421-line journal-guided proof rose to 770.948 seconds. A complete search-tree composition produced a

Case	Computation	Bytes	Initial Stage 5	Principal final abstraction
1	Prime-factor count; later singleton array wrapper benchmark	1348 / 1938	238.557 s scalar baseline; 1964.130 s array benchmark median	Direct-call annotations and singleton allocation/result theorem
2	First-match lookup in ten key–value pairs	7336	3260.962 s	Fixed search-chain wrapper and complete starter
3	Lookup in a fixed seven-node binary tree	7186	2427.376 s	Fixed search-tree wrapper and complete starter
4	Wrapping add-one map for arrays of length at most eight	1913	2364.735 s	Parameterized map wrapper, whole-function annotation, and complete starter

Table 1: Case-study subjects and original direct-proof times. Demo 1's scalar retained walkthrough and later array benchmark are distinct frozen artifacts.

326.320-second three-run median, and the later complete starter produced a 121.976-second median from 109.607, 121.976, and 131.413 seconds, with the same 77-line proof and no agent edit.

Demo 4 initially matched no proof region and received no recipe. Its 457-line proof took 2364.735 seconds and 27 edited Lean checks. Generalizing the allocator theorem to accept unused trailing locals reduced a controlled reproof to 1191.695 seconds and 19 edited checks while preserving all frozen inputs.

The next iteration added the parameterized map theorem, exact whole-function annotation, checked region equality, and complete starter. The unchanged 66-line starter passed its first Lean check, and Stage 5 completed in 103.123 seconds: 66.734 seconds in the coding-agent session and 27.688 seconds in outer acceptance. This is a 95.6 percent reduction from the held-out baseline and a 91.3 percent reduction from the allocator-only iteration.

Case and variant	Stage 5	Proof lines	Runs	Qualification
Demo 2 original	3260.962 s	1639	1	Direct generated artifact proof
Demo 2 chain starter	110.332 s	76	3	Median; 0.547 s range
Demo 3 original	2427.376 s	1398	1	Direct generated artifact proof
Demo 3 tree starter	121.976 s	77	3	Median; 21.806 s range
Demo 4 held-out	2364.735 s	457	1	No matched annotation or recipe
Demo 4 allocator only	1191.695 s	463	1	One generalized semantic region
Demo 4 map starter	103.123 s	66	1	Same artifact; theorem designed after baseline

Table 2: Selected controlled proof-generation results. Stage 5 measures start to first outer-accepted proof, so it includes agent, Lean, and orchestration time.

These cases support three limited conclusions. Exact semantic composition can remove target-level work that an agent otherwise repeats, and deterministic starters can turn proof generation

into proof checking for recognized templates. Proof length alone does not predict elapsed time, as several shorter intermediate proofs took longer than their predecessors.

The measurements do not establish general proof-generation performance. Demos 2 and 3 directly informed their chain and tree compositions, and Demo 4's final theorem was developed after its held-out journal exposed the missing regions. A new out-of-sample loop, aliasing, ownership, nested-data, or multi-function workload is required to measure whether the current annotations and proof kit generalize without another artifact-specific development cycle.

7 Trust, limitations, and failure boundaries

7.1 Trusted components

The logical result trusts the selected Lean kernel, the definitions of the binary grammar and validity judgments, the pinned Talos semantics, the artifact specification, and explicit host assumptions. It also trusts ordinary file reading at the operational boundary where the verifier compares the supplied file with the embedded byte value. SHA-256 protects identity handling but is not used as a logical replacement for byte equality.

The compiler, extractor, intermediate representation, serializer, WAT renderer, Talos model emitter, Wasmtime, coding agent, annotations, recipes, and journal remain outside the final theorem's premises. Some of these tools generate Lean source that enters the proof, but Lean checks that source under the permitted import and axiom policy. A defective producer should yield a failed equality, failed proof, or theorem about an inadequate human-approved specification.

The declaration audit rejects admitted proofs and unexpected axioms. The package and release records pin Lean, Talos, proof-library sources, verifier sources, Node, `wasm-tools`, Wasmtime, and artifact identities. The draft release records source revision `febed96d02f7654a`, but its matching cold-checkout receipt remains pending; the later Demo 4 work described here also postdates that release record.

7.2 Talos fidelity

Talos is a Lean implementation of WebAssembly execution with weakest-precondition support, but this project has not proved it equivalent to the normative semantics. Mechanized WebAssembly projects such as WasmCert and WasmRef-Isabelle establish a stronger semantics-to-interpreter connection [24, 26, 27]. The current project instead uses profile-specific corpus comparison and states Talos in the trusted base.

The conformance run found six failures in `memory_grow.wast`. The pinned Talos implementation copies an imported memory into the importing module's store and applies the import declaration's maximum during growth, rather than preserving shared runtime identity and the exported instance's maximum. A memory exported with maximum five pages can therefore grow to six pages after import through a declaration permitting six.

All twenty artifact subjects have one memory and no imports, so this defect does not exercise their imported-entity path. Their theorems remain valid statements under the pinned Talos semantics, while the discrepancy blocks a broader WebAssembly claim and weakens semantic fidelity evidence beyond the accepted profile. A faithful import model requires shared runtime instances or a proved equivalence for the closed-module specialization.

7.3 Lean kernel qualification

Both workspaces pin Lean 4.31.0 at commit `68218e87...`. That version accepts an archived reproduction deriving falsehood through a hand-built inductive declaration and an expression-hash collision. The repository's release evidence records this defect instead of treating a successful build as sufficient assurance.

The current disposition uses a lexical audit for literal `addDecl` and `inductDecl` references in the project proof tree and its two local LeanExe imports. This audit does not examine dependencies, aliases, compiled declarations, elaborator internals, other environment-mutation APIs, or the kernel defect. Publication-quality evidence requires moving to a fixed kernel or adding an independent checker whose soundness and accepted theory are understood.

7.4 Specification and profile limits

The artifact theorem proves the formal statement that the package contains. It cannot establish that an agent's formalization matches the user's prose, and no current source certificate proves that the generated Lean program implements the formal specification. Human review of *expected*, runtime premises, invalid-input behavior, and postconditions remains necessary.

The current `leanexegen` interface accepts and returns flat `Array UInt64` values, although an early scalar demo remains in the repository. Nested arrays, variable-width elements, aliases, shared ownership, recursive public data, imports, multiple memories, reference types, floating point, and host effects require new representation predicates, accepted binary forms, validation rules, and semantic support. The wider LeanExe compiler accepts more internal data and control patterns than the generation interface exposes.

Task-context separation is an orchestration policy rather than a proved information-flow property. The headless agent runs under the host's filesystem sandbox, so a deployment that requires demonstrable source blindness must place each task in an isolated filesystem or container. This qualification does not alter the final artifact theorem, whose dependency audit excludes the source and compiler regardless of what informed proof search.

Many artifact theorems cover valid inputs under explicit allocation and memory bounds. Division traps, invalid-input behavior, stack depth, instruction cost, and repeated-call memory use are incomplete or absent from the current user-facing guarantee set. Bump-only programs can leak across repeated calls by design, so a theorem cannot claim constant memory without a runtime or compiler change.

8 Related work

Table 3 locates the system among mechanized semantics, target program logics, verified compilers, per-run validation, and native-code checkers. The closest match depends on the boundary under discussion: program logics resemble the behavioral theorem, translation validation resembles the proposed source bridge, and proof-carrying systems resemble the published package. None of these comparisons removes the need to state the exact bytes, semantics, and host premises covered by each result.

8.1 Mechanized WebAssembly semantics and program logics

WebAssembly's design included a formal small-step semantics and type system from its first publication [7]. Watt's Isabelle mechanization produced a verified interpreter and type checker and

System or line	Established property	Relation to LeanExe
WasmCert and WasmRef [26, 27]	Mechanized WebAssembly semantics, soundness results, and a verified interpreter	Stronger semantics foundation; not an application-specific exact-byte package
Wasm Logic and Iris-Wasm [18, 25]	Functional correctness, separation, and modular safety for WebAssembly programs	Closest target-level program-logic comparison
CertiCoq-Wasm [13]	Verified compilation from CertiCoq's intermediate language to WebAssembly	Proves the compiler route rather than reproving each target specification
seL4 binary validation [19]	Per-build relation from verified source-level kernel to compiled binary	Closest precedent for the source-theorem transport roadmap
Self-certifying Wasm [15]	Independently checked evidence for compiler optimizations	Similar producer–checker split; evidence concerns optimization correctness
VeriISLE and VeriWasm [8, 22]	Native instruction-selection rules or sandbox isolation	Operate below the `.wasm` functional-theorem boundary
LeanExe	Behavioral predicate over the validated Talos module decoded from embedded bytes	Exact artifact identity and user-selected behavior under explicit premises

Table 3: Verification boundaries in related systems. The table compares established properties rather than implementation size or feature coverage.

found specification issues before standardization [24]. WasmCert-Isabelle and WasmCert-Coq later mechanized WebAssembly 1.0 in two proof assistants, while WasmRef-Isabelle refined the Isabelle semantics to an interpreter fast enough for use as a Wasmtime fuzzing oracle [26, 27].

SpecTec generates prose, executable definitions, and proof-assistant definitions from a common domain-specific source [32]. LeanExe currently maintains a separate restricted decoder, validator, and Talos semantics, which creates both implementation cost and semantic trust. Reusing or relating those components to a maintained mechanization would reduce this gap, but the present theorem chain still needs an exact binary-to-semantics boundary.

Wasm Logic provides a sound separation logic for first-order encapsulated WebAssembly and verifies a B-tree library in Isabelle/HOL [25]. Iris-Wasm supports modular higher-order reasoning, host composition, and safety against adversarial modules in Coq [18]. LeanExe uses Talos weakest preconditions and application-specific representation predicates; it handles exact binary identity and generated-runtime details, but its no-import profile and closed-module theorems do not provide Iris-Wasm's adversarial linking results.

8.2 Verified compilation and translation validation

CompCert proves semantic preservation from a substantial C source language to assembly across verified compilation passes [10]. CakeML verifies a functional-language compiler backend through multiple intermediate languages to concrete machine code [20]. CertiCoq-Wasm supplies a verified WebAssembly backend from CertiCoq's administrative-normal-form language using WasmCert-Coq [13].

Those systems amortize compiler proofs across every accepted source program. LeanExe currently pays a target-proof cost per artifact or emitted template and does not establish general source-to-target semantic preservation. Its artifact theorem can verify code without source and can

express a property narrower or richer than source equivalence, which makes it complementary to a verified compiler rather than a substitute.

Translation validation checks each compiler run instead of proving the compiler once [17]. Alive2 applies bounded, SMT-based translation validation to LLVM transformations and has found both optimizer and language-reference defects [12]. LeanExe artifact verification does not compare source and target at all; the proposed source-theorem transport phase would add a checked source-to-IR certificate and verified lowering, making that future path closer to proof-producing translation validation.

Translation validation for the seL4 kernel connects source-level correctness to the compiled binary by validating the compiler output against the semantics used by the proof [19]. A self-certifying WebAssembly compilation framework instead emits independently checked evidence for each optimization [15]. Both approaches provide closer precedents for the planned source-theorem transport layer than the current artifact-only theorem, whose specification is reproved directly at the target level.

Trust analyses of verified compilers emphasize that a semantic-preservation theorem does not eliminate specification, parser, assembler, proof-assistant, or operational tool boundaries [14]. The exact-byte design addresses one such boundary by making the binary the theorem subject. Its own Talos and Lean qualifications show that moving the boundary does not remove the need to account for every trusted definition and checker.

8.3 Machine-assisted proof generation

Lean combines a dependent type theory, an extensible elaborator, and metaprogrammable tactics in one implementation [6]. Aesop demonstrates configurable, white-box best-first proof search over registered rules [11]. These tools motivate a checked proof kit whose tactics expose stable semantic boundaries and whose applications remain ordinary Lean terms.

LeanDojo couples programmatic Lean interaction with retrieval over accessible premises, while DeepSeek-Prover studies whole-proof generation from large synthetic Lean corpora [30, 31]. The generation workflow uses a general coding agent with file editing and repeated Lean feedback rather than a specialized theorem-proving model. Its distinctive system problem is choosing exact artifact regions, memory representations, and compiler-template lemmas; the evaluation shows that deterministic checked composition can matter more than unconstrained proof search.

COPRA executes tactics, observes proof states and errors, and carries that feedback through a stateful theorem-proving search, while Pantograph exposes structured machine-to-machine Lean interaction [1, 21]. `leanexegen` uses the same basic feedback principle through ordinary files and constrained commands, with an outer acceptance check after the agent finishes. LEGO-Prover's growing-library design and evidence that purported library learning can collapse into single-use lemmas motivate this project's insistence on shared theorem statements and held-out artifact tests [3, 23].

8.4 Downstream native-code checking

WebAssembly execution usually includes another compiler from validated bytecode to native code. VeriISLE verifies instruction-selection rules used by Cranelift, while VeriWasm checks software-fault-isolation properties of native code compiled from WebAssembly [8, 22]. These systems address properties below the `.wasm` boundary; combining them with an artifact behavior theorem would still require a proved connection through the selected runtime and its remaining compilation passes.

9 Roadmap

The immediate release work is concrete. The recorded draft release needs its matching cold-checkout receipt, and a publication snapshot that includes the later proof-generation work needs a refreshed immutable release record. The repository also needs a fixed Lean kernel or independently justified checker, repeated measurements for the final Demo 4 method, and a new out-of-sample workload that exercises different loop, allocation, or ownership structure.

The next proof-generation phase should extend vertical annotation slices one semantic region at a time. Each slice should include compiler emission, sidecar validation, exact decoded matching, Lean equality generation, recipe selection, a deterministic starter or verification condition, end-to-end artifact proof, and fixed-input timing. Promotion should depend on proof time and acceptance rate rather than proof length or local tactic elegance.

A generated verification-condition system can move more work out of free-form agent editing. Its first useful fragment would handle straight-line instructions and branches, stop at calls and loops, accept explicit summaries and invariants, and emit labeled Lean obligations. Checked compiler annotations or a source-aware agent could propose invariants, while sound target-level rules would continue to derive the artifact theorem.

The artifact profile should expand only with corresponding grammar, validation, translation, semantics, and conformance evidence. Imports require repairing Talos runtime identity or moving to a semantics that models shared instances. Nested and shared heap values require ownership, alias, frame, and ABI theorems before `leanexegen` can expose richer public types.

Cost and resource guarantees require semantic instrumentation. Step counts, maximum call depth, operand-stack depth, and heap watermarks should become explicit observables with generic composition theorems. A runtime change that resets or reuses allocation state between calls must precede a steady-state memory theorem.

Source-theorem transport remains a separate optional result. The proposed path freezes a proof-grade intermediate representation, checks a source-to-IR certificate that mentions the original Lean function, proves lowering to a target module, and equates that module with the validated translation of exact artifact bytes. A generic theorem can then transport a user's source predicate through the ABI without weakening the independent artifact-only workflow.

Full compiler verification would generalize that transport from generated certificates to all accepted declarations and lowering passes. CompCert, CakeML, and CertiCoq-Wasm show the strength and cost of that destination. Direct artifact proofs should remain useful for third-party binaries, compiler-independent review, and properties whose statement begins at the deployed machine interface.

10 Conclusion

LeanExe demonstrates an exact-artifact proof boundary for a restricted class of WebAssembly modules. The final theorem starts from embedded bytes, passes through proved-sound decoding and validation, and reaches an application behavior theorem through checked equality with a Talos execution model. Source code, compilation, annotation production, and agent reasoning can all fail without becoming logical assumptions.

The proof-generation experiments identify a condition for scaling this boundary. Reusable theorems must capture semantic compiler templates, and annotations must select those theorems through exact artifact checks. Complete compositions reduced Demos 2 and 3 to unchanged starters and reduced the held-out Demo 4 proof from 2364.735 to 103.123 seconds after the missing loop

abstraction was developed.

The result remains bounded by its evidence. The final Demo 4 intervention has no independent out-of-sample confirmation, Talos fidelity is assumed and has a known imported-memory defect outside the profile, and Lean 4.31.0 has a recorded kernel defect. Resolving those boundaries, broadening the artifact profile, and separating deterministic verification conditions from agent search are prerequisites for a publication claim stronger than this implementation report.

A Reproduction record

The manuscript describes repository snapshot `04f4170a141ef7bd`, which contains the reported Demo 4 results. The compiler and proof workspaces pin Lean 4.31.0, the proof workspace pins Talos revision `bb3277e21c9786e3`, and the conformance configuration pins Wasmtime 44.0.0, `wasm-tools` 1.251.0, and official testsuite revision `9233a0a8d5920a8d`. The draft release record identifies the earlier source revision `febed96d02f7654a`; a final submission needs an immutable release record and matching cold-checkout receipt for the manuscript snapshot.

The exact-artifact aggregate command is `tools/artifact-proof.js check-all`, and the semantic comparison command is `tools/artifact-conformance.js check`. Independent generated-package checking uses `tools/leanexegen verify <package>`. Repository policy routes every Lean or Lake child through the serialized, resource-limited `tools/leanrun` command.

The retained demo inputs, artifacts, specifications, programs, proofs, telemetry, annotations, recipes, journals, and walkthroughs reside under `demos/`. Fixed-artifact Demo 1 timing packages reside under `benchmarks/leanexegen/demo1-array/`, and the chronological measurements and failed variants appear in `devnotes.md`. The artifact registry and content-addressed binaries reside under `proofs/artifacts/`, while the decoder, validator, translation, behavioral proofs, runtime library, and proof kit reside under `proofs/talos/lean/Project/`.

References

[1] Leni Aniva, Chuyue Sun, Brando Miranda, Clark Barrett, and Sanmi Koyejo. Pantograph: A machine-to-machine interaction interface for advanced theorem proving, high level reasoning, and data extraction in Lean 4. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 15696 of *Lecture Notes in Computer Science*, pages 104–123. Springer, 2025. doi: 10.1007/978-3-031-90643-5_6.

[2] Andrew W. Appel. Foundational proof-carrying code. In *Proceedings of the 16th Annual IEEE Symposium on Logic in Computer Science*, pages 247–256. IEEE, 2001. doi: 10.1109/LICS.2001.932501.

[3] Ian Berlot-Attwell, Frank Rudzicz, and Xujie Si. Library learning doesn't: The curious case of the single-use "library". *arXiv preprint arXiv:2410.20274*, 2024. doi: 10.48550/arXiv.2410.20274.

[4] Bytecode Alliance. Wasmtime: A standalone runtime for WebAssembly. Software and documentation, 2026. URL `https://docs.wasmtime.dev/`. LeanExe evaluation version 44.0.0.

[5] Cajal Technologies. Talos: A WebAssembly interpreter and verification framework in Lean. Software repository, 2026. URL `https://github.com/cajal-technologies/talos`. LeanExe evaluation revision bb3277e21c9786e3133d5c1601e34ebdc0bea4df.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[7] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 185–200. Association for Computing Machinery, 2017. doi: 10.1145/3062341.3062363.

[8] Evan Johnson, David Thien, Yousef Alhessi, Shravan Narayan, Fraser Brown, Sorin Lerner, Tyler McMullen, Stefan Savage, and Deian Stefan. Doveryai, no proveryai: SFI safety for native-compiled Wasm. In *Proceedings of the Network and Distributed System Security Symposium*. Internet Society, 2021. doi: 10.14722/ndss.2021.24078.

[9] LeanExe contributors. LeanExe: Compilation and exact-artifact verification for Lean and WebAssembly. Software repository, 2026. URL `https://github.com/jsmorph/leanexe`. Manuscript snapshot 04f4170a141ef7bd289f81bc739f66d07471900d.

[10] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7): 107–115, 2009. doi: 10.1145/1538788.1538814.

[11] Jannis Limperg and Asta Halkjær From. Aesop: White-box best-first proof search for Lean. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 253–266. Association for Computing Machinery, 2023. doi: 10.1145/3573105.3575671.

[12] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 65–79. Association for Computing Machinery, 2021. doi: 10.1145/3453483.3454030.

[13] Wolfgang Meier, Martin Jensen, Jean Pichon-Pharabod, and Bas Spitters. CertiCoq-Wasm: A verified WebAssembly backend for CertiCoq. In *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 127–139. Association for Computing Machinery, 2025. doi: 10.1145/3703595.3705879.

[14] David Monniaux and Sylvain Boulmé. The trusted computing base of the CompCert verified compiler. *arXiv preprint arXiv:2201.10280*, 2022. doi: 10.48550/arXiv.2201.10280.

[15] Kedar S. Namjoshi and Anton Xue. A self-certifying compilation framework for WebAssembly. In *Verification, Model Checking, and Abstract Interpretation*, volume 12597 of *Lecture Notes in Computer Science*, pages 127–148. Springer, 2021. doi: 10.1007/978-3-030-67067-2_7.

[16] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119. Association for Computing Machinery, 1997. doi: 10.1145/263699.263712.

[17] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 1384 of *Lecture Notes in Computer Science*, pages 151–166. Springer, 1998. doi: 10.1007/BFb0054170.

[18] Xiaojia Rao, Aïna Linn Georges, Maxime Legoupil, Conrad Watt, Jean Pichon-Pharabod, Philippa Gardner, and Lars Birkedal. Iris-Wasm: Robust and modular verification of WebAssembly programs. *Proceedings of the ACM on Programming Languages*, 7(PLDI), 2023. doi: 10.1145/3591265.

[19] Thomas Arthur Leck Sewell, Magnus O. Myreen, and Gerwin Klein. Translation validation for a verified OS kernel. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 471–482. Association for Computing Machinery, 2013. doi: 10.1145/2491956.2462183.

[20] Yong Kiam Tan, Magnus O. Myreen, Ramana Kumar, Anthony Fox, Scott Owens, and Michael Norrish. The verified CakeML compiler backend. *Journal of Functional Programming*, 29:e2, 2019. doi: 10.1017/S0956796818000229.

[21] Amitayush Thakur, George Tsoukalas, Yeming Wen, Jimmy Xin, and Swarat Chaudhuri. An in-context learning agent for formal theorem-proving. In *Proceedings of the First Conference on Language Modeling*, 2024. URL `https://openreview.net/forum?id=V7HRrxXUhN`.

[22] Alexa VanHattum, Monica Pardeshi, Chris Fallin, Adrian Sampson, and Fraser Brown. Lightweight, modular verification for WebAssembly-to-native instruction selection. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 231–248. Association for Computing Machinery, 2024. doi: 10.1145/3617232.3624862.

[23] Haiming Wang, Huajian Xin, Chuanyang Zheng, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Enze Xie, Jian Yin, Zhenguo Li, and Xiaodan Liang. LEGO-Prover: Neural theorem proving with growing libraries. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=3f5PALef5B`.

[24] Conrad Watt. Mechanising and verifying the WebAssembly specification. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 53–65. Association for Computing Machinery, 2018. doi: 10.1145/3167082.

[25] Conrad Watt, Petar Maksimović, Neelakantan R. Krishnaswami, and Philippa Gardner. A program logic for first-order encapsulated WebAssembly. In *33rd European Conference on Object-Oriented Programming*, volume 134 of *Leibniz International Proceedings in Informatics*, pages 9:1–9:30. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2019. doi: 10.4230/LIPIcs.ECOOP.2019.9.

[26] Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, and Philippa Gardner. Two mechanisations of WebAssembly 1.0. In *Formal Methods: 24th International Symposium, FM 2021*, volume 13047 of *Lecture Notes in Computer Science*, pages 61–79. Springer, 2021. doi: 10.1007/978-3-030-90870-6_4.

[27] Conrad Watt, Maja Trela, Peter Lammich, and Florian Märkl. WasmRef-Isabelle: A verified monadic interpreter and industrial fuzzing oracle for WebAssembly. *Proceedings of the ACM on Programming Languages*, 7(PLDI):100–123, 2023. doi: 10.1145/3591224.

[28] WebAssembly Community Group. Official WebAssembly specification testsuite. Software repository, 2026. URL `https://github.com/WebAssembly/testsuite`. LeanExe evaluation revision 9233a0a8d5920a8d32358ee915a3662ff3385029.

[29] WebAssembly Working Group. *WebAssembly Core Specification, Release 3.0*, 2026. URL `https://webassembly.github.io/spec/core/`. Binary format version 1.

[30] Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*, 2024. doi: 10.48550/arXiv.2405.14333.

[31] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Animashree Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.52202/075280-0944.

[32] Dongjun Youn, Joachim Breitner, Philippa Gardner, Jaehyun Lee, Sam Lindley, Matija Pretnar, Xiaojia Rao, Andreas Rossberg, Sukyoung Ryu, Wonho Shin, and Conrad Watt. Bringing the WebAssembly standard up to speed with SpecTec. *Proceedings of the ACM on Programming Languages*, 8(PLDI), 2024. doi: 10.1145/3656440.